

Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the **Alpha-Beta Pruning algorithm** to optimize the Minimax search in game playing.

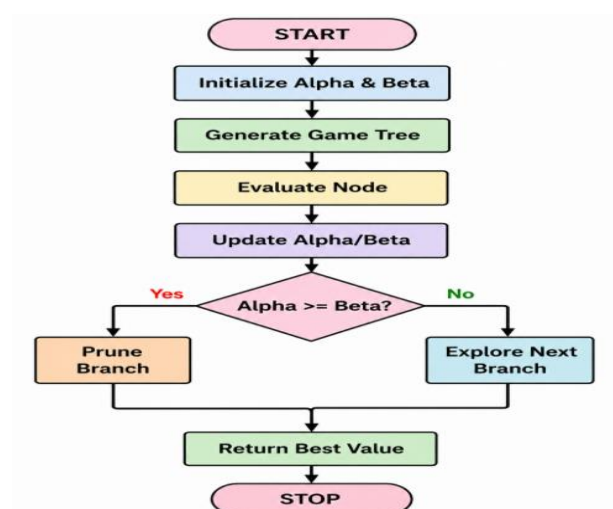
Objectives

- To understand Alpha-Beta pruning.
- To reduce unnecessary game-tree exploration.
- To improve the efficiency of Minimax.
- To find the optimal game decision.

Algorithm

1. Start.
2. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
3. Generate the game tree.
4. Evaluate the current node.
5. For maximizing nodes, update α .
6. For minimizing nodes, update β .
7. If $\alpha \geq \beta$, prune the remaining branches.
8. Return the best value.
9. Stop.

Flowchart



PROGRAM

```

def alphabeta(depth, node, alpha, beta, maximizing):
    if depth == 0:
        return node

    if maximizing:
        value = -999

        for child in [node + 2, node + 3]:
            value = max(
                value,
                alphabeta(depth - 1, child, alpha, beta, False)
            )

            alpha = max(alpha, value)

            if alpha >= beta:
                break

        return value

    else:
        value = 999

        for child in [node + 2, node + 3]:
            value = min(
                value,
                alphabeta(depth - 1, child, alpha, beta, True)
            )

            beta = min(beta, value)

            if alpha >= beta:
                break

        return value

result = alphabeta(3, 0, -999, 999, True)
print("Best Value:", result)

```

OUTPUT

```

= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python39-64/Python.exe
Best Value: 8

```

Result

The Alpha-Beta Pruning algorithm was successfully implemented and unnecessary branches were eliminated during the search.